

Reinforcement Learning

Chapter 3: Model-free RL

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Fall 2026

Practical Setting: *Model-free RL*

We now assume that **do not know** the **MDP** that describes **environment**

Model-Based RL

Bellman Equation

value iteration

policy iteration

Model-free RL

on-policy methods

temporal difference

Monte Carlo

SARSA

off-policy methods

Q-learning

- + If this is the **typical case**, why did we spend so much time on **MDPs**?
- We are going to do **the same thing only without explicit model**

Generalized Policy Iteration

We wrote the pseudo-code for GPI as below

```
GenPolicyItr():  
  1: Initiate two random policies  $\pi$  and  $\bar{\pi}$   
  2: while  $\pi \neq \bar{\pi}$  do  
  3:    $v_\pi = \text{GenPolicyEval}(\pi)$  and  $\pi \leftarrow \bar{\pi}$   
  4:    $\bar{\pi} = \text{PolicyImprov}(v_\pi)$   
  5: end while
```

Let's recall where we had to use **environment's model**

- ① In **policy evaluation** when we compute values via **Bellman equations**
- ② In **policy improvement** when we compute **action-values** out of **values**
- ? How can we do this **without** knowing the **MDP**?

Recap: Computing Statistics from Data

We have an **unknown** generator which returns X at **random** $\rightsquigarrow$ we see

$$Y = f(X)$$

for some function $f(\cdot)$

We want to know the **expected** output Y

$$\mu_Y = \mathbb{E}\{Y\} = \mathbb{E}\{f(X)\}$$

If we knew the **generation model**, we could write

$$\mu_Y = \mathbb{E}\{f(X)\} = \sum_{\underbrace{x \in \mathbb{X}}_{\text{all outcomes}}} f(x) \underbrace{p(x)}_{\text{model}}$$

Recap: Computing Statistics from Data

- + What can we do if we **don't know** the **model** $p(x)$?
- We could use **Monte-Carlo method**

In **Monte-Carlo method**, we sample K times **independently** as

$$Y_1, Y_2, \dots, Y_K$$

Then we **estimate** the **expected** value as

$$\hat{\mu}_Y = \frac{1}{K} \sum_{k=1}^K Y_k$$

Monte-Carlo Method

- + Why does Monte-Carlo work?
- Simply because of **central limit theorem**

If $Y_1, Y_2, \dots, Y_K$ are **independent** samples of **identical process**, we could say

$$\hat{\mu}_Y \sim \mathcal{N}\left(\mu_Y, \frac{\sigma^2}{K}\right)$$

when K is **large enough**: so we could think of it as

$$\hat{\mu}_Y \approx \mu_Y + \frac{\varepsilon}{\sqrt{K}}$$

for some random **error** term ε : this error vanishes as K goes **large**

Computing Values via Monte-Carlo

? But, how can we apply this idea to RL?

💡 Recall that **values** and **action-values** are **expected** returns!

Say we want to compute **value of state s** for **policy π** in an **episodic** environment
~~~~> Monte-Carlo suggest that

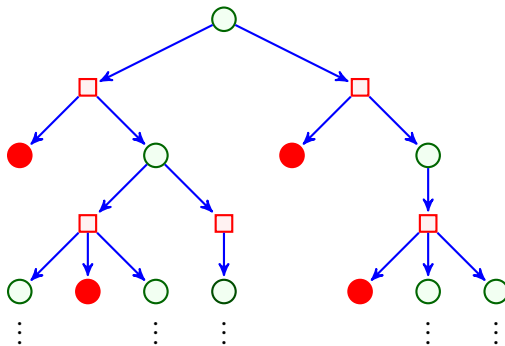
- 1 We start at **state  $s$**  and play with **policy  $\pi$**  until we meet **terminal state**  
↳ Say it happens at **time  $T$**
- 2 We compute **sample return** as  $G[1] = R_1 + \gamma R_2 + \dots + \gamma^{T-1} R_T$
- 3 We repeat this for  $K$  episodes and each episode, we collect  $G[k]$

$$\text{estimate of value} = \hat{v}_{\pi}(s) = \frac{1}{K} \sum_{k=1}^K G[k]$$

# Values via Monte-Carlo: *Trajectory Sampling*

We can look at it as estimating values from *sample trajectories*

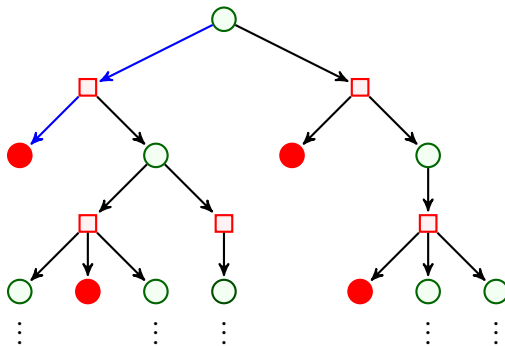
➔ With MDP, we *average* over all *possible trajectories*



## Values via Monte-Carlo: *Trajectory Sampling*

We can look at it as estimating values from *sample trajectories*

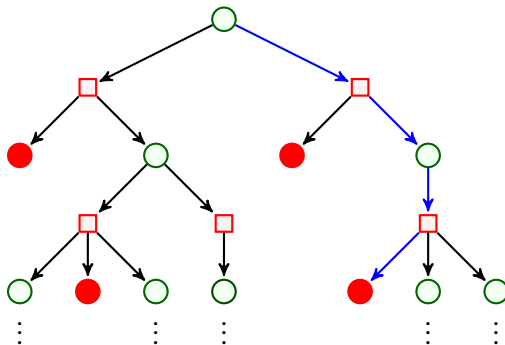
➔ *Without* MDP, we *sample* trajectories and *estimate the average*



# Values via Monte-Carlo: *Trajectory Sampling*

We can look at it as estimating values from *sample trajectories*

➔ *Without* MDP, we *sample* trajectories and *estimate the average*



# Computing Values via Monte-Carlo: Algorithm 1

MC\_verI( $\pi, s$ ):

1: Initialize estimator of value as  $\hat{v}_{\pi}(s) = 0$

2: **for** episode = 1 :  $K$  **do**

3:   Initialize with **state**  $S_0 = s$  and act via policy  $\pi(a|s)$

4:   Sample a trajectory

$$S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \dots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T: \text{terminal}$$

5:   Compute sample return  $G = R_1 + \gamma R_2 + \dots + \gamma^{T-1} R_T$

6:   Update estimate of value as  $\hat{v}_{\pi}(s) \leftarrow \hat{v}_{\pi}(s) + G/K$

7: **end for**

# Computing Values via Monte-Carlo

- + Doesn't that take *too long* to compute a single value?
- This is a *general problem*, but our algorithm is also too *naive*!

---

In our algorithm, we need to sample *all  $K$  episodes* to access an *estimate*, but we rather prefer to have a *bad estimate* which *gradually improves* over episodes

We could use the idea of *online averaging*  $\equiv$  *incremental averaging*

Let's find out what it is!

## Online Averaging

Say, we want to compute the average of  $K$  samples: we could write

$$\begin{aligned}\eta_K &= \frac{1}{K} \sum_{k=1}^K G_k = \frac{1}{K} \left( \sum_{k=1}^{K-1} G_k + G_K \right) = \frac{1}{K} ((K-1) \eta_{K-1} + G_K) \\ &= \left( 1 - \frac{1}{K} \right) \eta_{K-1} + \frac{G_K}{K} \\ &= \eta_{K-1} + \frac{1}{K} (G_K - \eta_{K-1})\end{aligned}$$

But, we can define the previous average as

$$\eta_{K-1} = \frac{1}{K-1} \sum_{k=1}^{K-1} G_k \rightsquigarrow \sum_{k=1}^{K-1} G_k = (K-1) \eta_{K-1}$$

# Online Averaging: Geometric Weights

## Online Averaging

We can update the average in *online fashion* as

$$\eta_K = \eta_{K-1} + \frac{1}{K} \Delta_K$$

where  $\Delta_K = G_K - \eta_{K-1}$  is the deviation in  $K$ -th episode

The above expression is given for *uniform averaging weights*, i.e., all samples have same weights: in more general form, we could update as

$$\eta_K = \eta_{K-1} + \alpha \Delta_K$$

for some  $0 < \alpha \leq 1$  that can be *fixed* or *scaled with  $K$*

- if it is *fixed*  $\equiv$  computing weighted average with *geometric* weights
- if it is *scaled linearly with  $K$*   $\equiv$  computing *linear* averaging

# Computing Values via Monte-Carlo: Algorithm II

MC\_verII( $\pi, s$ ):

- 1: Initialize estimator of value as  $\hat{v}_\pi(s) = 0$
- 2: **for** episode = 1 :  $K$  **do**
- 3:   Initialize with **state**  $S_0 = s$  and act via policy  $\pi(a|s)$
- 4:   Sample a trajectory

$$S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \dots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T: \text{terminal}$$

- 5:   Compute sample return  $G = R_1 + \gamma R_2 + \dots + \gamma^{T-1} R_T$
- 6:   Update estimate of value as  $\hat{v}_\pi(s) \leftarrow \hat{v}_\pi(s) + \alpha(G - \hat{v}_\pi(s))$
- 7: **end for**

Now *after each episode*, we have an estimate of value function at **state**  $s$

# Computing Values via Monte-Carlo: *Improve Efficiency*

! We go over *one complete trajectory* to estimate *value of a single state!*

💡 We can estimate values of *other states* down the *trajectory!*

*In the following sample trajectory*

$$S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \dots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

*it's not only  $S_0$  whose sample return can be computed! We can also compute sample returns of  $S_1, \dots, S_{T-1}$*

$$\begin{aligned} G(S_1) &= R_2 + \gamma R_3 + \dots + \gamma^{T-2} R_T \\ &\vdots \\ G(S_{T-1}) &= R_T \end{aligned}$$

# All-Visit Monte-Carlo: Algorithm III

This concludes a policy evaluation algorithm based on [Monte-Carlo](#)

MC\_Eval( $\pi$ ):

- 1: Initialize estimator of value as  $\hat{v}_\pi(s^n) = 0$  for  $n = 1 : N$
- 2: **for** episode = 1 :  $K$  **do**
- 3:   Initialize with a **random state**  $S_0$  and act via policy  $\pi(a|s)$
- 4:   Sample a trajectory

$$S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \dots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T: \text{terminal}$$

- 5:   Initialize with  $G = 0$
- 6:   **for**  $t = T - 1 : 0$  **do**
- 7:     Update current return  $G \leftarrow R_{t+1} + \gamma G$
- 8:     Update estimate of value as  $\hat{v}_\pi(S_t) \leftarrow \hat{v}_\pi(S_t) + \alpha(G - \hat{v}_\pi(S_t))$
- 9:   **end for**
- 10: **end for**

# All-Visit Monte-Carlo: Convergence

It's *intuitive* to say this algorithm converges to true values after *lots of episodes*

## Asymptotic Convergence of Monte-Carlo

Let  $\mathcal{C}_K(s)$  denote number of visits at *state*  $s$  during  $K$  Monte-Carlo episodes. Assume that the random state initialization is distributed such that  $\mathcal{C}_K(s^n)$  *grows large as  $K$  increases* for  $n = 1 : N$ , i.e.,

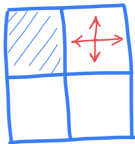
$$\lim_{K \rightarrow \infty} \mathcal{C}_K(s^n) = \infty$$

Then, as  $K \rightarrow \infty$  the value estimators converge to the exact values, i.e.,

$$\hat{v}_\pi(s) \xrightarrow{K \uparrow \infty} v_\pi(s)$$

for any *state*  $s$

## Example: Dummy Grid World with Random Walk



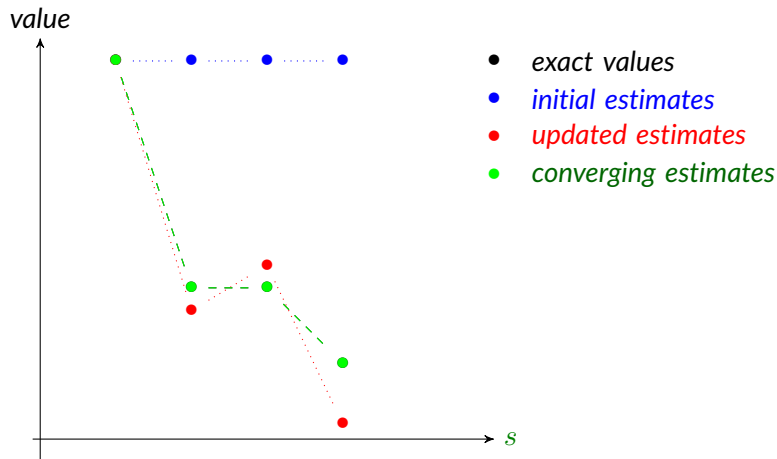
Let's get back to our *dummy world*: we now use *Monte-Carlo method* to compute the values for *uniform random policy*, i.e.,

$$\pi(a|s) = \frac{1}{4}$$

for all *actions* and *states*. From *Bellman equations*, we have

$$v_{\pi}(0) = 0 \quad v_{\pi}(1) = -4.5 \quad v_{\pi}(2) = -4.5 \quad v_{\pi}(3) = -6$$

## Example: *Dummy Grid World with Random Walk*

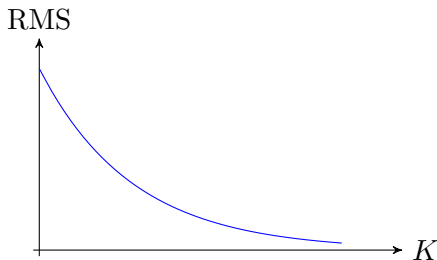


## Typical Behavior: Variation Against Number of Episodes

The error of our estimate in each *episode* is

$$\text{RMS} = \sqrt{\sum_{n=1}^N |\hat{v}_{\pi}(s^n) - v_{\pi}(s^n)|^2}$$

If we plot it against  $K$ ; then, we see

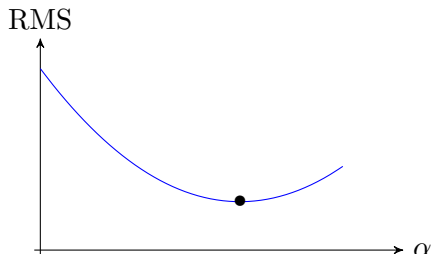


## Typical Behavior: Variation Against Averaging Coefficient

The error of our estimate in each *episode* is

$$\text{RMS} = \sqrt{\sum_{n=1}^N |\hat{v}_{\pi}(s^n) - v_{\pi}(s^n)|^2}$$

If we plot it against  $\alpha$ ; then, we could see a *minimum*



# Monte-Carlo Method: Action-Values

- + Now that we have *Monte-Carlo algorithm*, can we use it in GPI?
- Not yet! Remember that we need *action-values* for *policy improvement*

Recall that *action-values* are defined as

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} \{G_t | s, a\}$$

We can again use *Monte-Carlo* to estimate *action-values* as

$$\hat{q}_{\pi}(s, a) = \frac{1}{K} \sum_{k=1}^K G[k] \leftarrow \text{starting at } s \text{ with action } a$$

# All-Visit Monte-Carlo: Action-Values

MC\_QEval( $\pi$ ):

- 1: Initialize estimator as  $\hat{q}_\pi(s^n, a^m) = 0$  for  $n = 1 : N$  and  $m = 1 : M$
- 2: **for** episode = 1 :  $K$  **do**
- 3:     Initialize with a random state-action pair  $(S_0, A_0)$  and act via policy  $\pi(a|s)$
- 4:     Sample a trajectory

$$S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \dots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T: \text{terminal}$$

- 5:     Initialize with  $G = 0$
- 6:     **for**  $t = T - 1 : 0$  **do**
- 7:         Update current return  $G \leftarrow R_{t+1} + \gamma G$
- 8:         Update  $\hat{q}_\pi(S_t, A_t) \leftarrow \hat{q}_\pi(S_t, A_t) + \alpha(G - \hat{q}_\pi(S_t, A_t))$
- 9:     **end for**
- 10: **end for**

We can now apply GPI using the *Monte-Carlo method*!

# Policy Iteration with Monte-Carlo

Starting with a random **policy** we can use GPI

- 1 We evaluate the **policy**
  - ↳ We use **Monte-Carlo method** to estimate **action-values**
- 2 We improve by selecting best **action** for each **state**
  - ↳ We typically call this **greedy** improvement

```
MC_PolicyItr():
```

```
1: Initiate two random policies  $\pi$  and  $\bar{\pi}$ 
```

```
2: while  $\pi \neq \bar{\pi}$  do
```

```
3:    $\hat{q}_{\pi} = \text{MC\_QEval}(\pi)$  and  $\pi \leftarrow \bar{\pi}$ 
```

```
4:    $\bar{\pi} \leftarrow \text{Greedy}(\hat{q}_{\pi})$ 
```

```
5: end while
```

# Policy Iteration with Monte-Carlo

Algorithmically, we can write the greedy update as

Greedy( $\hat{q}_\pi$ ):

1: **for**  $n = 1 : N$  **do**

2:   Improve the policy by taking *deterministically* the *best action*

$$\bar{\pi}(a^m | s^n) = \begin{cases} 1 & m = \underset{m}{\operatorname{argmax}} \hat{q}_\pi(s^n, a^m) \\ 0 & m \neq \underset{m}{\operatorname{argmax}} \hat{q}_\pi(s^n, a^m) \end{cases}$$

3: **end for**

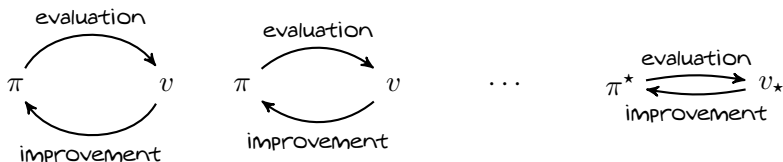
This is *not* however the *best* we could do!

We are going to have a whole lecture about it

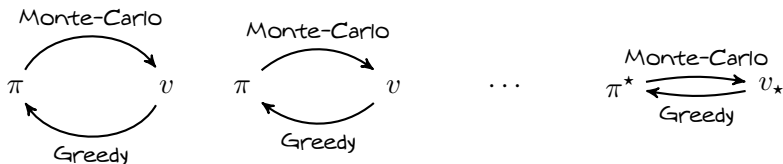
We get back to this point in Section 4

# GPI with Monte-Carlo

For any GPI, we said that we can think of

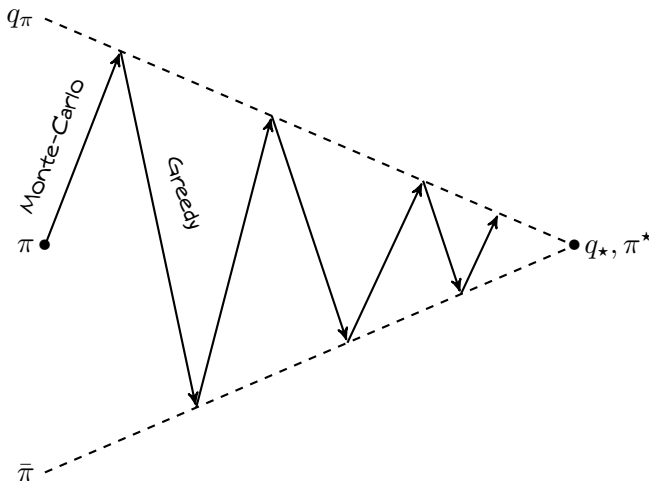


With **Monte-Carlo** evaluation, we can show this procedure as



## GPI with Monte-Carlo

Another way to visualize this procedure is to think of following diagram



# Non-Episodic Monte-Carlo: *Terminating Trajectory*

- + We only discussed *episodic* scenarios! Don't we use model-free RL in *non-episodic environment*?
- Sure we do! But, *Monte-Carlo* is not the best approach

A basic idea in this case is to *terminate sample trajectories*

$$S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \dots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

- With *long enough*  $T$  and  $\gamma < 1$  the very later terms are *ineffective*
- But, we *cannot* use all the states in the trajectories
  - ↳ Sample returns of those who are *close to time*  $T$  are *not reliable*!

$$G_{T-1} = R_T + \underbrace{\gamma R_{T+1} + \dots}_{\text{we terminated them!}}$$

# Terminating Monte-Carlo

**TerminMC\_Eval( $\pi$ ):**

- 1: Initialize estimator of value as  $\hat{v}_\pi(s^n) = 0$  for  $n = 1 : N$
- 2: Choose **very large  $T$  and  $W$**  that satisfy  $W < T$
- 3: **for** episode = 1 :  $K$  **do**
- 4:   Initialize with a **random state  $S_0$**  and act via policy  $\pi(a|s)$
- 5:   Sample a trajectory and terminate after  **$T$  time steps**

$$S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \dots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T: \text{terminated}$$

- 6:   Initialize with  $G = 0$
- 7:   **for**  $t = T - 1 : 0$  **do**
- 8:     Update current return  $G \leftarrow R_{t+1} + \gamma G$
- 9:     **if**  $t < T - W$  **then**
- 10:       Update estimate of value as  $\hat{v}_\pi(S_t) \leftarrow \hat{v}_\pi(S_t) + \alpha(G - \hat{v}_\pi(S_t))$
- 11:     **end if**
- 12:   **end for**
- 13: **end for**